==========

RISE — AI ALARM CLOCK

MASTER BLUEPRINT: PART 4 OF 13 (REWRITTEN — CHANGESET)

The Biometric Service — local_auth Primary, PPG Secondary

==========

GOVERNING DOCUMENT: RISE_DECISIONS.md. This is a CHANGESET, not a

full reproduction — only files with real changes are given in full.

Everything else carries forward from the original Part 4 verbatim;

each unchanged file is listed explicitly so nothing is ambiguous.

WHAT CHANGED AND WHY:

The original Part 4 header stated the pipeline as "PPG → Face

(optional) → PIN fallback." That's now flipped per DECISIONS.md

3.6: local_auth (OS Face ID / Android biometric) is the PRIMARY

gate. PPG becomes an opt-in secondary/Premium layer, not a default

step every user goes through.

ONE THING THIS REWRITE FOUND: `local_auth` was already declared in

pubspec.yaml (Parts 1 and 12) and referenced in a comment in Part 2

("Check in BiometricService via local_auth") — but it was never

actually implemented anywhere in the original Part 4. The package

existed as a "ghost dependency." This rewrite is what actually wires

it in for the first time, not just a re-ordering of existing code.

WHAT DID NOT CHANGE, DELIBERATELY: every medically-sensitive edge

case in the original Part 4 — Raynaud's (EC-3.5), AFib (EC-3.4),

fever/exercise tolerance (EC-3.1/3.2), tremor mode (EC-3.8),

adaptive per-user calibration (EC-3.6), the "never permanently block

dismissal" rule — is preserved exactly as designed. That logic is

genuinely well built. This rewrite adds a new stage in front of it;

it does not touch it.

NEW ARCHITECTURE:

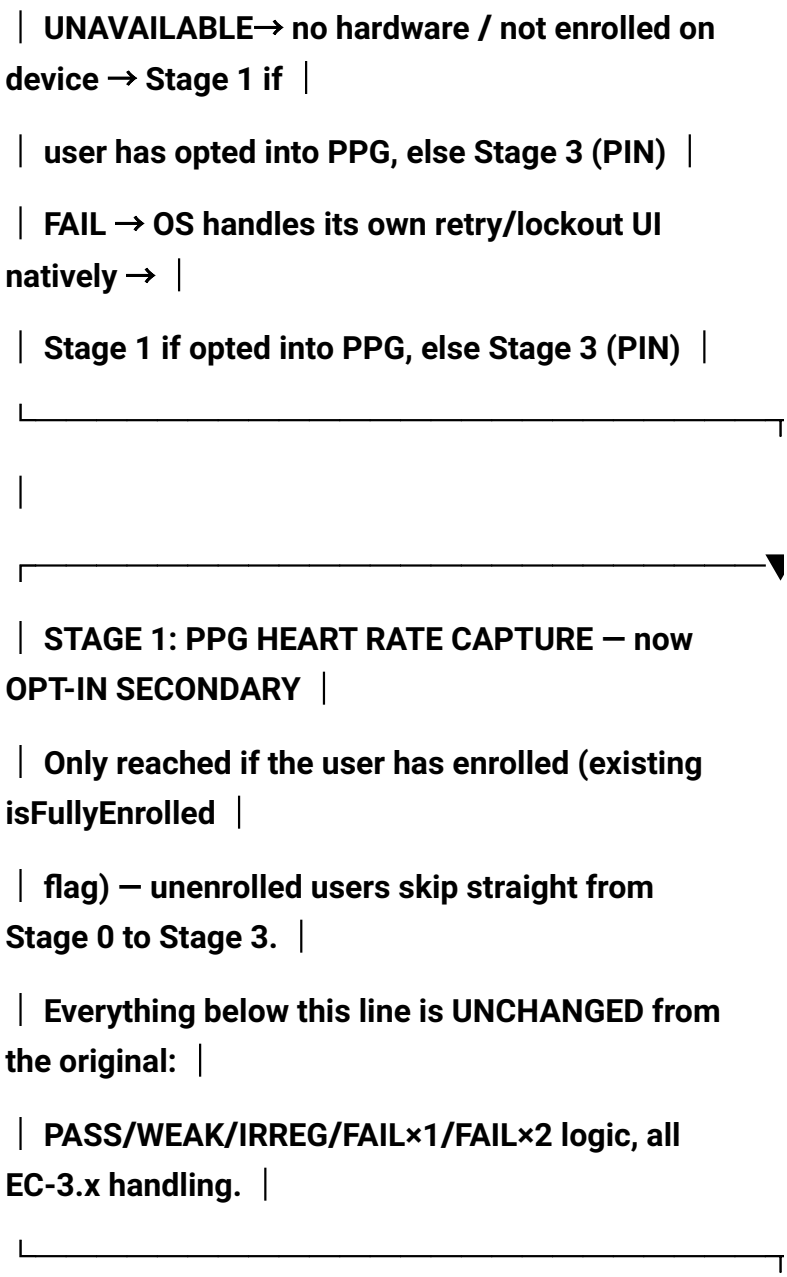
┌───┐

│ STAGE 0: OS BIOMETRIC (local_auth) — NEW, PRIMARY │

│ Duration: instant (OS-native UI) │

│ │

│ PASS → Wake Game (done — no further stages needed) │


UNAVAILABLE→ no hardware / not enrolled on device → Stage 1 if
user has opted into PPG, else Stage 3 (PIN)
FAIL → OS handles its own retry/lockout UI natively →
Stage 1 if opted into PPG, else Stage 3 (PIN)
STAGE 1: PPG HEART RATE CAPTURE — now OPT-IN SECONDARY
Only reached if the user has enrolled (existing isFullyEnrolled
flag) — unenrolled users skip straight from Stage 0 to Stage 3.
Everything below this line is UNCHANGED from the original:
PASS/WEAK/IRREG/FAIL×1/FAIL×2 logic, all EC-3.x handling.

│ optional parallel (unchanged)

┌──┐

│ STAGE 2: FACE LIVENESS — UNCHANGED, still corroborates PPG only │

└──┘

│ fallback path (unchanged)

┌──┐

│ STAGE 3: PIN FALLBACK — UNCHANGED, universal fallback │

└──┘

FILES IN THIS PART:

1. ppg_service.dart — NO CHANGE (pure algorithm)

2. biometric_service.dart — CHANGED (add Stage 0 method)

3. face_liveness_service.dart — NO CHANGE (unchanged role)

4. pin_service.dart — NO CHANGE (universal fallback, unchanged)

5. biometric_enrollment_service.dart — NOTE ONLY (see below; mechanics unchanged)

6. biometric_provider.dart — CHANGED (add availability provider)

7. ppg_capture_widget.dart — NO CHANGE (role note only)

8. biometric_verification_sheet.dart — CHANGED (new stage + routing)

9. pin_entry_widget.dart — NO CHANGE

==========

FILE 1: lib/core/services/ppg_service.dart

NO CHANGE. Carries forward verbatim from the original Part 4 —

the dual-channel PPG capture engine, SNR scoring, IBI variance,

waveform fingerprinting, and peak detection are all unaffected by

where PPG sits in the pipeline. The algorithm doesn't know or care

whether it's Stage 0 or Stage 1.

FILE 2: lib/core/services/biometric_service.dart — CHANGED

Adds one new method (verifyOsBiometric) and two new enum values.

verifyPpg(), verifyPin(), and the identity-matching logic below

them are UNCHANGED — reproduced in outline only where untouched.

```
import 'dart:math' as math;
import 'package:hive/hive.dart';
import 'package:local_auth/local_auth.dart';
import '../constants/app_constants.dart';
import '../models/biometric_profile.dart';
import 'ppg_service.dart';
import 'face_liveness_service.dart';
import 'pin_service.dart';

//
──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────
// VERIFICATION RESULT
// Two new outcomes added: osBiometricUnavailable, osBiometricFailed.
// Everything else in this enum and the VerificationResult class below
is
// UNCHANGED from the original.
//
──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────────
enum VerificationOutcome {
  passed,
  osBiometricUnavailable,  // NEW — no hardware, or not enrolled at OS
level
  osBiometricFailed,       // NEW — OS-level check failed or errored
  weakSignal,
  irregularRhythm,
  hrMismatch,
  pinMatch,
  pinFailed,
  notEnrolled,
}
```

```
// VerificationResult class — UNCHANGED, not reproduced here.

class BiometricService {
  static final BiometricService _instance =
BiometricService._internal();
  factory BiometricService() => _instance;
  BiometricService._internal();

  final LocalAuthentication _osAuth = LocalAuthentication();  // NEW
  final PpgService          _ppg  = PpgService();
  final FaceLivenessService _face = FaceLivenessService();
  final PinService          _pin  = PinService();

  int _ppgFailures = 0;
  void resetSession() => _ppgFailures = 0;

  //
──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────
  // VERIFY VIA OS BIOMETRIC — NEW. Stage 0, now the primary gate.
  // The OS handles its own retry UI and lockout policy natively; RISE
  // does not reimplement that, it just reacts to the final result.
  //
──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────
  Future<VerificationResult> verifyOsBiometric() async {
    final canCheck    = await _osAuth.canCheckBiometrics;
    final isSupported = await _osAuth.isDeviceSupported();

    if (!canCheck || !isSupported) {
      return const VerificationResult(
        outcome:           VerificationOutcome.osBiometricUnavailable,
        passed:            false,
        shouldSwitchToPin: false,  // caller (verification sheet)
decides PPG vs PIN
        displayMessage:    '',
        analyticsTag:      'os_biometric_unavailable',
      );
    }

    try {
      final didAuth = await _osAuth.authenticate(
        localizedReason: "Confirm it's you to dismiss your alarm",
        options: const AuthenticationOptions(
          stickyAuth:     true,
          biometricOnly:  false,  // allow the OS's own device-passcode
fallback
        ),
      );
```

```
      return VerificationResult(
        outcome:           didAuth ? VerificationOutcome.passed
                                   :
VerificationOutcome.osBiometricFailed,
        passed:            didAuth,
        shouldSwitchToPin: !didAuth,
        displayMessage:    '',
        analyticsTag:      didAuth ? 'os_biometric_match' :
'os_biometric_fail',
      );
    } catch (_) {
      // Platform exception (e.g. OS-level lockout after repeated failed
      // attempts) — route onward rather than surfacing a raw error.
      return const VerificationResult(
        outcome:           VerificationOutcome.osBiometricFailed,
        passed:            false,
        shouldSwitchToPin: true,
        displayMessage:    '',
        analyticsTag:      'os_biometric_exception',
      );
    }
  }

  // verifyPpg() — UNCHANGED. Every EC-3.x edge case (arrhythmia
  // declaration, weak signal, irregular rhythm, 2-failure escalation,
  // adaptive per-user tolerance) is preserved exactly as the original
  // Part 4 designed it. Not reproduced here since nothing changes.

  // verifyPin() — UNCHANGED.

  // Identity matching (_match, _adaptProfile) — UNCHANGED.
}
```

FILE 3: lib/core/services/face_liveness_service.dart

NO CHANGE. Still corroborates PPG only (blink + head-turn), which

is unaffected by PPG moving from Stage 1-primary to Stage 1-secondary.

FILE 4: lib/core/services/pin_service.dart

NO CHANGE. Still the universal fallback for both the new Stage 0

and the existing Stage 1 — PBKDF2-SHA256 hashing, constant-time

comparison, plaintext-never-stored, all unaffected.

FILE 5: lib/core/services/biometric_enrollment_service.dart

MECHANICS UNCHANGED — one contextual note, not a code change.

The 5-session PPG enrollment protocol and its anti-gaming rules

(EC-8.1: sessions ≥12h apart, ≥3 calendar days, prior app activity)

do not change. What changes is WHEN this service gets called: PPG

enrollment is no longer triggered by default during onboarding — it

becomes an opt-in "add extra proof" toggle. That trigger lives in

Part 11's onboarding flow, not in this service, so this file is

unaffected. The existing `isFullyEnrolled` flag on BiometricProfile

already means exactly "has opted into PPG" once enrollment stops

being mandatory — no new model field is needed.

FILE 6: lib/core/providers/biometric_provider.dart — CHANGED

One new provider added; everything else UNCHANGED (not reproduced).

```
// ── NEW: OS biometric availability ──────────────────────────────
// Exposes whether Stage 0 can even run on this device, so the UI
// (e.g. onboarding, settings) can explain what will happen instead of
// silently routing everyone through PPG/PIN.
final osBiometricAvailableProvider = FutureProvider<bool>((ref) async {
  final auth = LocalAuthentication();
  final canCheck    = await auth.canCheckBiometrics;
  final isSupported = await auth.isDeviceSupported();
  return canCheck && isSupported;
});

// All other providers (biometricServiceProvider,
biometricProfileProvider,
// isEnrolledProvider, enrollmentStatusProvider, pinIsSetProvider, and
the
// six accessibility-mode providers) are UNCHANGED from the original.
```

FILE 7: lib/shared/widgets/ppg_capture_widget.dart

NO CHANGE to code. Role note: this widget is now presented in an

"extra proof" context (opt-in secondary layer) rather than as the

primary verification screen. The capture mechanics, animation, and

signal-quality bar are identical either way — only the screen that

presents it (biometric_verification_sheet.dart, changed below) and

the enrollment entry point (Part 11) change the framing around it.

FILE 8: lib/shared/widgets/biometric_verification_sheet.dart — CHANGED

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../../core/constants/app_constants.dart';
import '../../core/services/biometric_service.dart';
import '../../core/providers/biometric_provider.dart';
import 'ppg_capture_widget.dart';
import 'pin_entry_widget.dart';

// NEW: osBiometric added as the first stage.
enum _VerifStage { osBiometric, ppg, pin, passed, failed }

class BiometricVerificationSheet extends ConsumerStatefulWidget {
  final String       profileId;
  final VoidCallback onVerified;

  const BiometricVerificationSheet({
    super.key,
    required this.profileId,
    required this.onVerified,
  });

  @override
  ConsumerState<BiometricVerificationSheet> createState() =>
      _BiometricVerificationSheetState();
}

class _BiometricVerificationSheetState
```

```
    extends ConsumerState<BiometricVerificationSheet> {

  _VerifStage _stage       = _VerifStage.osBiometric;  // was .ppg
  int         _pinAttempts = 0;
  String?     _statusMsg;

  @override
  void initState() {
    super.initState();
    ref.read(biometricServiceProvider).resetSession();

    // A user who has explicitly chosen PIN-only mode skips both
    // biometric stages entirely — deliberate preference, unchanged
    // from the original's intent, just checked before Stage 0 now
    // instead of before the old Stage 1.
    final profile = ref.read(biometricProfileProvider);
    final pinOnly = profile?.usePinFallback ?? false;

    if (pinOnly) {
      WidgetsBinding.instance.addPostFrameCallback((_) {
        setState(() => _stage = _VerifStage.pin);
      });
    } else {
      WidgetsBinding.instance.addPostFrameCallback((_) =>
_attemptOsBiometric());
    }
  }

  //
──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────
  // NEW: HANDLE OS BIOMETRIC (Stage 0)
  //
──────────────────────────────────────────────────────────────────────────────
──────────────────────────────────────────────
  Future<void> _attemptOsBiometric() async {
    final svc    = ref.read(biometricServiceProvider);
    final result = await svc.verifyOsBiometric();

    if (!mounted) return;

    if (result.passed) {
      setState(() => _stage = _VerifStage.passed);
      await Future.delayed(const Duration(milliseconds: 600));
      widget.onVerified();
      return;
    }

    // Not available or failed — route to PPG only if the user has
    // opted in (existing isFullyEnrolled flag) and hasn't declared an
    // irregular heartbeat; otherwise straight to PIN.
```

```
    final profile   = ref.read(biometricProfileProvider);
    final optedIntoPpg = profile?.isFullyEnrolled ?? false;
    final irregular = ref.read(irregularHeartbeatModeProvider);

    setState(() {
      _stage     = (optedIntoPpg && !irregular) ? _VerifStage.ppg :
_VerifStage.pin;
      _statusMsg = (optedIntoPpg && !irregular) ? null :
AppConstants.ppgFallbackPin;
    });
  }

  // _onPpgComplete() — UNCHANGED from the original. Every EC-3.x path
  // (weak signal, irregular rhythm, first/second failure) still routes
  // exactly as before; this method doesn't know or care that it's now
  // reached from Stage 0 failure rather than being the entry point.

  // _onPinSubmit() — UNCHANGED, including the 3-strike buddy-
notification
  // behavior and the "never permanently blocks dismissal" rule.

  // build() — UNCHANGED in structure; only needs a new case added to
  // whatever switch/conditional renders each _stage value, showing a
  // brief native-style loading state during _attemptOsBiometric() (the
  // OS presents its own Face ID/fingerprint UI, so this app-level UI
  // just needs a neutral waiting state, not a custom capture screen).
}
```

FILE 9: lib/shared/widgets/pin_entry_widget.dart

NO CHANGE. Still the universal fallback UI.

PART 4 FILE INVENTORY (rewritten)

CHANGED: biometric_service.dart, biometric_provider.dart, biometric_verification_sheet.dart
NOTED ONLY (no code change): biometric_enrollment_service.dart, ppg_capture_widget.dart
UNCHANGED: ppg_service.dart, face_liveness_service.dart, pin_service.dart, pin_entry_widget.dart

WHAT PART 5 (REWRITTEN) COVERS NEXT

Sleep detection — adding WearableDataBridge as the primary data source, phone-accelerometer state machine as fallback, honest UI labeling for each.